REACT FITNESS TRACKER

SYSTEM ARCHITECTURE & TECHNICAL PLANNING DOCUMENT

Umuzi (BBD internship)

Frontend Engineering Capstone Specification

September 2026

1. Executive Project Overview

A simple, client-side fitness tracker and workout planner built for gym-goers who want an easy way to stay consistent.

Discover exercise routines, build your own weekly workout plans, log your sets and progress, watch form videos, listen to motivational tracks, and keep an eye on your key fitness metrics — all in one place.

Made with React (functional components + hooks), modular CSS, React Router, and browser storage so your data sticks around. Fully tested with Jest and React Testing Library.

2. Component Architecture Hierarchy

The application architecture adheres to a 3+ tier nested component hierarchy structured around reusable atomic UI controls, modular layout components, and centralized route-level page views.

```
App.jsx (Root Layout & State Hub)
  ├── Navbar (Sticky Header & Active Link Styling)
  ├── Routes (React Router Container)
  │     ├── Home (Hero Banner, Summary Callouts, AudioPlayer)
  │     ├── ExercisesPage (SearchBar, ExerciseFilter, ExerciseList)
  │     │     └── ExerciseList ──> ExerciseCard (Badge, Button)
  │     ├── ExerciseDetail (VideoPlayer, Instructions, Modal, Badge)
  │     ├── WorkoutPlannerPage ──> WorkoutPlanner
  │     │     └── DayCard (x7 Days) ──> ExerciseCard, Button
  │     ├── HistoryPage ──> WorkoutLog
  │     │     └── LogEntry (Sets, Reps, Weight Metrics)
  │     ├── ProgressPage ──> ProgressChart (Calorie & Streak Summary)
  │     └── NotFound (404 Fallback View)
  └── Footer (Global Links & Attribution)
```

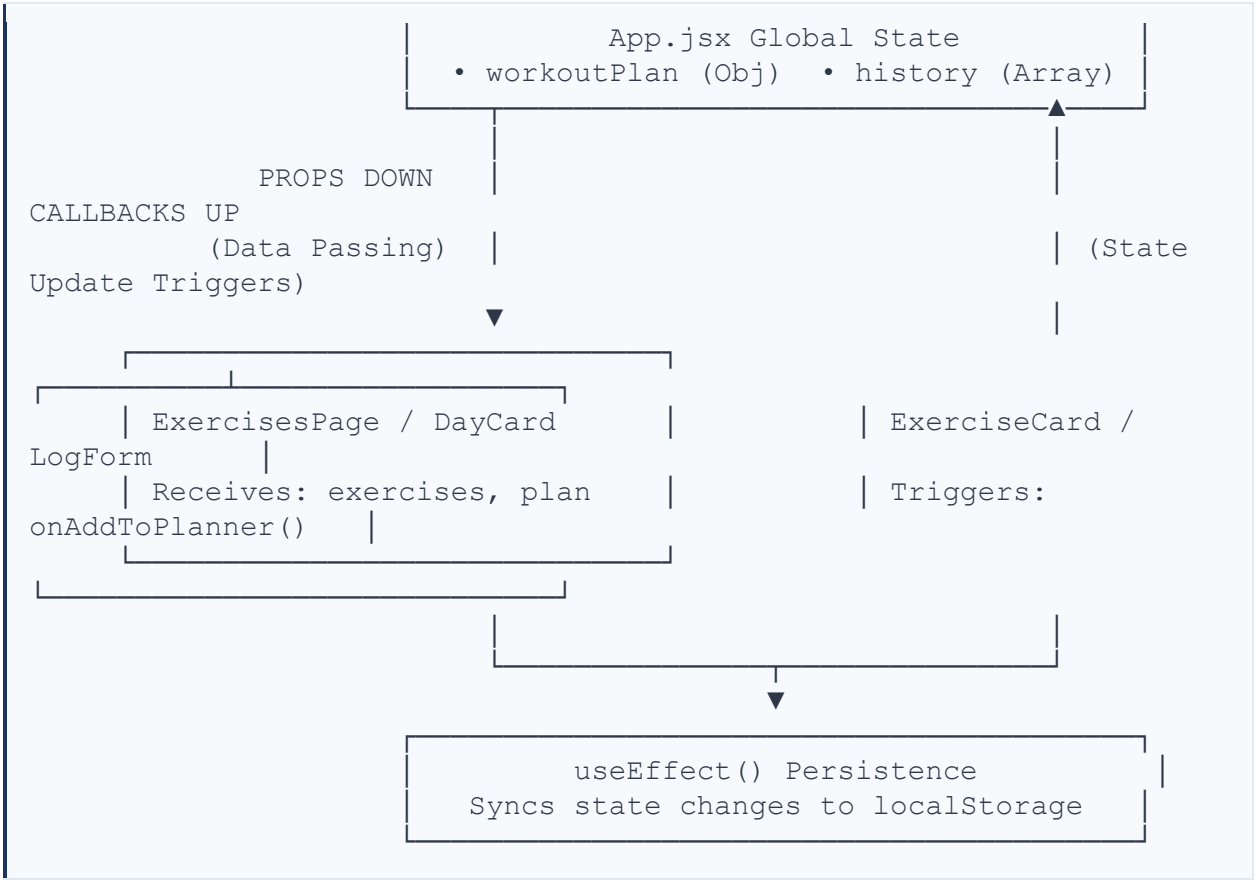
Component Inventory & Specification

Component	Type / Layer	Key Props Passed	Primary Role
App	Root Hub	None (Top-Level)	Holds global state, routing shell, localStorage effects.
Navbar	Navigation	activeRoute, onNavigate	Sticky header links with active route

			indicators.
ExerciseCard	Reusable Leaf	exercise, onSelect, onAdd	Displays thumbnail, tags, difficulty badge, action buttons.
DayCard	Planner Container	day, workouts, onRemove	Displays 1 of 7 days; accepts dropped or added exercises.
LogEntry	History Item	log, onDelete	Renders historical workout session details (sets, reps, weight).
VideoPlayer	Media Feature	videoUrl, title, controls	HTML5 video player with fallback browser message.
AudioPlayer	Media Feature	audioUrl, title, autoplay	HTML5 audio player driving workout motivation tracks.
Button	Atomic UI	variant, onClick, children	Reusable CTA button supporting primary, secondary, danger.
Modal	Atomic UI	isOpen, onClose, children	Reusable overlay container using children prop composition.
Badge	Atomic UI	type, label, colorStyle	Visual tag display for exercise categories & difficulties.

3. Data Flow & State Management Strategy

Shared data lives in one place (App.jsx) so every page stays in sync. Changes flow up through simple callbacks, and the app automatically saves everything to the browser’s local storage, so nothing gets lost.



State Variable Specifications

State Variable	Type / Default	Location	Purpose / Storage Sync
exercises	Array []	App.jsx	Main repository of exercise catalog fetched from exercisesData.js.
workoutPlan	Object {mon..sun}	App.jsx	7-day schedule mapping array of exercises. Syncs to localStorage('workoutPlan').
workoutHistory	Array []	App.jsx	Logs completed exercises with reps/sets/weight. Syncs to localStorage('workoutHistory').
searchTerm	String ''	ExercisesPage	Local input state controlling dynamic search filtering.
selectedCategory	String 'all'	ExercisesPage	Filter selection for category (Strength,

			Cardio, Flexibility, Balance).
currentLog	Object {...}	HistoryPage	Form input state capturing sets, reps, weight, and date before logging.

4. Multi-Approach Styling Architecture

To demonstrate modularity, responsiveness, and style isolation, three styling strategies are implemented across the codebase:

- **CSS Modules (*.module.css):** Used for primary component styling (Navbar, ExerciseCard, DayCard) providing scoped, collision-free CSS classes.
- **Inline Styles (style={{...}}):** Applied for dynamic calculations such as progress bar width percentages, custom RGB category badge background fills, and active indicator positions.
- **Conditional Styling:** Dynamic class name toggling based on boolean component state (e.g., active route tabs, completed workout checks, difficulty level borders).

5. Comprehensive Testing Strategy & Matrix

Testing is implemented using Jest and React Testing Library (@testing-library/react).

Test Suite Category	Min Target	Focus Area	Key Execution Examples
Component Tests	5+ Tests	Isolated Rendering	ExerciseCard renders details; Badge reflects correct difficulty prop.
Integration Tests	4+ Tests	Cross-Module Flow	Adding exercise from ExercisesPage renders inside WorkoutPlanner DayCard.
User Interaction	3+ Tests	Events & Callbacks	userEvent.type in SearchBar triggers onSearch callback with input term.
Routing Tests	2+ Tests	Router Behavior	Navigating to invalid path renders 404 NotFound page with home redirect.
Hook & Async Tests	4+ Tests	Effects & Storage	useLocalStorage syncs state; waitFor handles simulated exercise data fetch.